PROGRAMAÇÃO ORIENTADA A OBJETOS COM JAVA

Encapsulamento, Construtores, Getters e Setters

Nesta aula serão aprofundados os mecanismos usados para proteger o estado dos objetos, controlar o acesso aos atributos e criar instâncias consistentes.

Encapsulamento

Proteção e controle do estado interno.

Construtores

Definição do estado inicial do objeto.

Getters e Setters

Consulta, alteração e validação de dados.

1. RETOMADA

O ponto de partida

Na aula anterior foram estudados classe, objeto, atributo, método, instanciação e construtor.

```
public class Aluno {
    String nome;
    int idade;
}

Aluno aluno = new Aluno();
aluno.nome = "Maria";
aluno.idade = 20;
```

> O problema é que qualquer parte do sistema poderia executar `aluno.idade = -50;`.

2. CONCEITO PRINCIPAL

O que é encapsulamento?

Encapsulamento é o princípio que protege o estado interno de um objeto e controla como seus dados podem ser consultados ou modificados.

Sem controle

```
public double preco;
produto.preco = -1000;
```

Com encapsulamento

```
private double preco;
```

Outras classes deixam de alterar o atributo diretamente.

3. VISIBILIDADE

Modificadores de acesso

Modificador	Acesso
`public`	Qualquer classe com acesso ao objeto.
`private`	Somente dentro da própria classe.
`protected`	Classe, pacote e subclasses.
sem modificador	Classes do mesmo pacote.

> Nesta aula o foco principal será `public` e `private`.

4. PROTEÇÃO DE ESTADO

Atributos privados

```
public class Aluno {
    private String nome;
    private int idade;
}
```

Depois dessa alteração, `aluno.nome = "Maria";` não será permitido fora da classe.

> O acesso deverá ocorrer por meio de métodos públicos definidos pela própria classe.

5. CONSULTA

Getter

Getter é um método usado para consultar um atributo privado.

```
public String getNome() {
    return nome;
}
```

```
System.out.println(aluno.getNome());
```

6. ALTERAÇÃO

Setter

Setter é um método utilizado para alterar um atributo de maneira controlada.

```
public void setNome(String nome) {
    this.nome = nome;
}
```

```
aluno.setNome("Maria");
```

7. ACESSO CONTROLADO

Getter e Setter juntos

```
public class Aluno {
    private String nome;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
}
```

```
aluno.setNome("Maria");
System.out.println(aluno.getNome());
```

8. CONSISTÊNCIA

Validação utilizando Setter

```
public void setIdade(int idade) {
    if (idade >= 0) {
        this.idade = idade;
    } else {
        System.out.println("Idade inválida.");
    }
}
```

> A classe passa a impedir que o objeto entre em um estado inválido.

9. ORGANIZAÇÃO

Quem deve validar?

Regra fora da classe

```
if (idade >= 0) {
    pessoa.idade = idade;
}
```

A regra pode ser esquecida ou duplicada.

Regra dentro da classe

```
pessoa.setIdade(idade);
```

A própria classe centraliza a regra.

10. EXEMPLO

Classe Produto encapsulada

```
public class Produto {
    private String nome;
    private double preco;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public double getPreco() {
        return preco;
    }

    public void setPreco(double preco) {
        if (preco >= 0) {
            this.preco = preco;
        }
    }
}
```

11. UTILIZAÇÃO

Manipulando Produto

```
Produto produto = new Produto();

produto.setNome("Notebook");
produto.setPreco(3500);

System.out.println(produto.getNome());
System.out.println(produto.getPreco());
```

> O código externo não precisa saber como os dados estão armazenados internamente.

12. INICIALIZAÇÃO

O construtor

O construtor é executado quando uma nova instância é criada.

```
Aluno aluno = new Aluno();
```

> O trecho `Aluno()` representa a chamada ao construtor.

13. CONSTRUÇÃO EXPLÍCITA

Criando um construtor

```
public class Aluno {
    private String nome;
    private int idade;

    public Aluno(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }
}
```

```
Aluno aluno = new Aluno("Maria", 20);
```

14. ESTADO INICIAL

Por que utilizar construtores?

Sem construtor parametrizado

```
Aluno aluno = new Aluno();
aluno.setNome("Maria");
aluno.setIdade(20);
```

Com construtor

```
Aluno aluno =
    new Aluno("Maria", 20);
```

O construtor permite exigir dados essenciais no momento da criação.

15. CONSISTÊNCIA

Evitando objetos incompletos

Dependendo do sistema, um aluno sem nome, matrícula ou curso pode não representar um objeto válido.

```
Aluno aluno = new Aluno(
    "Maria",
    "2026001",
    "Sistemas para Internet"
);
```

> O construtor define quais informações são obrigatórias para a criação.

16. REFERÊNCIA AO OBJETO

A palavra-chave this

```
public Aluno(String nome) {
    this.nome = nome;
}
```

this.nome

Atributo do objeto atual.

nome

Parâmetro recebido pelo construtor.

17. CONSISTÊNCIA

Construtor com validação

```
public Produto(String nome, double preco) {
    this.nome = nome;

    if (preco >= 0) {
        this.preco = preco;
    }
}
```

O construtor também pode impedir estados inválidos.

18. EVITANDO DUPLICAÇÃO

Reutilizando o Setter no construtor

```
public Produto(String nome, double preco) {
    this.nome = nome;
    setPreco(preco);
}
```

> A mesma regra de validação é reutilizada durante a criação e nas alterações posteriores.

19. COMPORTAMENTO DO JAVA

Construtor padrão

Quando nenhum construtor é declarado, Java oferece automaticamente um construtor sem parâmetros.

```
public class Pessoa {
    private String nome;
}

Pessoa pessoa = new Pessoa();
```

> Ao declarar um construtor parametrizado, o construtor vazio deixa de ser criado automaticamente.

20. ATENÇÃO

Construtor parametrizado e construtor vazio

```
public Pessoa(String nome) {
    this.nome = nome;
}
```

Nesse caso, o código abaixo deixa de ser válido:

```
Pessoa pessoa = new Pessoa();
```

Será necessário usar o construtor existente ou declarar também um construtor sem parâmetros.

21. DIFERENTES FORMAS DE CRIAR

Sobrecarga de construtores

```
public Pessoa() { }

public Pessoa(String nome) {
    this.nome = nome;
}

public Pessoa(String nome, int idade) {
    this.nome = nome;
    this.idade = idade;
}
```

A classe passa a oferecer várias formas válidas de criação.

22. IDENTIFICAÇÃO

Assinatura de métodos e construtores

Pessoa()

Sem parâmetros.

Pessoa(String nome)

Um parâmetro.

Pessoa(String nome, int idade)

Dois parâmetros.

Como o Java escolhe?

Pelos argumentos informados na chamada.

23. MESMO NOME

Sobrecarga de métodos

```
public class Calculadora {

    public int somar(int a, int b) {
        return a + b;
    }

    public double somar(double a, double b) {
        return a + b;
    }
}
```

```
calculadora.somar(10, 20);
calculadora.somar(10.5, 20.8);
```

24. ENCAPSULAMENTO REAL

Nem todo atributo precisa de Setter

Considere o saldo de uma conta bancária.

```
private double saldo;

public double getSaldo() {
    return saldo;
}
```

> É possível permitir a consulta sem permitir uma alteração direta por `setSaldo()`.

25. REGRAS DE DOMÍNIO

Controlando o saldo corretamente

```
public void depositar(double valor) {
    if (valor > 0) {
        saldo += valor;
    }
}

public void sacar(double valor) {
    if (valor > 0 && valor <= saldo) {
        saldo -= valor;
    }
}
```

O saldo muda somente por operações coerentes com a conta.

26. EXEMPLO COMPLETO

Classe ContaBancaria

```
public class ContaBancaria {
    private String titular;
    private int numero;
    private double saldo;

    public ContaBancaria(String titular, int numero) {
        this.titular = titular;
        this.numero = numero;
        this.saldo = 0;
    }

    public void depositar(double valor) {
        if (valor > 0) saldo += valor;
    }

    public void sacar(double valor) {
        if (valor > 0 && valor <= saldo) saldo -= valor;
    }

    public String getTitular() { return titular; }
    public int getNumero() { return numero; }
    public double getSaldo() { return saldo; }
}
```

27. EXECUÇÃO

Utilizando ContaBancaria

```
ContaBancaria conta =
    new ContaBancaria("Maria", 1234);

conta.depositar(1000);
conta.sacar(200);

System.out.println(conta.getTitular());
System.out.println(conta.getSaldo());
```

> Saída esperada: **Maria** e **800.0**.

28. INTERPRETAÇÃO

Encapsulamento não é apenas private

Consulta

Quais dados podem ser vistos?

Alteração

Quais dados podem ser modificados?

Validação

Quais valores são permitidos?

Comportamento

Quais operações fazem sentido para o objeto?

29. PROJETO INADEQUADO

Encapsulamento superficial

```
public class Conta {
    private double saldo;

    public void setSaldo(double saldo) {
        this.saldo = saldo;
    }
}
```

> Ainda seria possível executar `conta.setSaldo(-100000);`.

O uso de `private` precisa estar acompanhado de regras coerentes.

30. EXEMPLO PRÁTICO

Classe Funcionario

```
public class Funcionario {
    private String nome;
    private String cargo;
    private double salario;

    public Funcionario(
        String nome,
        String cargo,
        double salario
    ) {
        this.nome = nome;
        this.cargo = cargo;
        setSalario(salario);
    }
}
```

31. VALIDAÇÃO

Salário e aumento

```
public void setSalario(double salario) {
    if (salario >= 0) {
        this.salario = salario;
    }
}

public double getSalario() {
    return salario;
}

public void aumentarSalario(double percentual) {
    if (percentual > 0) {
        salario = salario +
                  salario * percentual / 100;
    }
}
```

32. EXECUÇÃO

Testando Funcionario

```
Funcionario funcionario =
    new Funcionario(
        "Carlos",
        "Desenvolvedor",
        3000
    );

System.out.println(funcionario.getSalario());
funcionario.aumentarSalario(10);
System.out.println(funcionario.getSalario());
```

> Saída esperada: **3000.0** e depois **3300.0**.

33. ATIVIDADE GUIADA

Classe Produto

Criar uma classe `Produto` com os atributos privados **nome**, **preço** e **quantidade**.

Adicionar construtor, getters e setters adequados.

Implementar `calcularValorEstoque()`, `adicionarEstoque()` e `removerEstoque()`.

34. ATIVIDADE GUIADA

Regras do Produto

- O preço não poderá ser negativo.
- A quantidade não poderá ser negativa.
- Não poderá ser removida quantidade maior que o estoque.
- Somente valores positivos poderão ser adicionados.

```
Produto produto =
    new Produto("Teclado", 150, 10);

produto.adicionarEstoque(5);
produto.removerEstoque(3);

System.out.println(produto.getQuantidade());
```

> Resultado esperado: **12**.

35. ATIVIDADE PRÁTICA INDIVIDUAL

Sistema simples de Veículo

Criar a classe `Veiculo` com os atributos privados:

marca, modelo, ano e velocidade.

O construtor deverá receber marca, modelo e ano. A velocidade inicial deverá ser zero.

36. COMPORTAMENTOS

Métodos da classe Veiculo

acelerar(double valor)

Aceitar somente valores maiores que zero.

frear(double valor)

Impedir velocidade abaixo de zero.

getVelocidade()

Consultar a velocidade atual.

Getters

Consultar marca, modelo e ano.

37. RESULTADO ESPERADO

Utilizando Veiculo

```
Veiculo carro =
    new Veiculo(
        "Honda",
        "Civic",
        2024
    );

carro.acelerar(50);
carro.frear(20);

System.out.println(carro.getVelocidade());
```

> Resultado esperado: **30.0**.

38. DESAFIO ADICIONAL

Velocidade máxima

```
private double velocidadeMaxima;
```

Modificar `acelerar()` para impedir que o veículo ultrapasse esse limite.

> Se a velocidade atual for 180 km/h, a máxima for 200 km/h e a aceleração solicitada for 50 km/h, o resultado deverá ser 200 km/h.

39. ATENÇÃO

Erros comuns

Esquecer o this

```
nome = nome;
```

O atributo não é alterado corretamente.

Acessar private diretamente

```
produto.preco = 100;
```

Não será permitido fora da classe.

Setter para tudo

Nem todo atributo deve ser alterável diretamente.

Duplicar validação

Regras devem ser centralizadas e reaproveitadas.

40. SÍNTESE

Como o objeto deve se comportar?

OBJETO → **ATRIBUTOS PRIVADOS** →

MÉTODOS PÚBLICOS → **REGRAS**

O objeto deve controlar seu próprio estado e oferecer ao restante do sistema apenas as operações necessárias.

41. COMPARAÇÃO

Sem encapsulamento × Com encapsulamento

Sem encapsulamento

```
produto.preco = -500;
```

Alteração direta e sem validação.

Com encapsulamento

```
produto.setPreco(-500);
```

A própria classe pode rejeitar a operação.

42. SÍNTESE DA AULA 02

Conteúdos consolidados

Encapsulamento

Proteção e controle do estado.

public / private

Controle de visibilidade.

Getters / Setters

Acesso e validação.

Construtores

Inicialização do objeto.

this

Referência ao objeto atual.

Sobrecarga

Mesmos nomes, parâmetros diferentes.

43. CONTINUIDADE

Relacionamentos entre objetos

Depois de compreender como uma classe controla seu estado, o próximo passo é estudar como diferentes objetos se relacionam.

Associação

Um objeto conhece ou utiliza outro.

Agregação

Uma parte pode existir independentemente do todo.

Composição

Uma relação forte entre objeto principal e suas partes.

Exemplos

Aluno → Curso, Pedido → Cliente e Pedido → Produto.